



Department of Artificial Intelligence and Machine Learning

Experiment No.: 5 To implement Splay Tree

Aim

To implement a **Splay Tree** and perform basic operations such as insertion, searching, deletion, and splaying of nodes.

Theory

A **Splay Tree** is a self-adjusting Binary Search Tree (BST) in which the most recently accessed node is moved to the root of the tree using a process called **splaying**. Unlike other self-balancing search trees such as AVL Trees and Red-Black Trees, a Splay Tree does not maintain explicit balance information such as height or color. The main idea behind a Splay Tree is that recently accessed elements are likely to be accessed again. Therefore, whenever a node is inserted, searched, or accessed, the tree is reorganized so that the accessed node is moved toward the root. This reduces the access time for frequently used elements.

A Splay Tree maintains the Binary Search Tree property:

- All keys in the left subtree of a node are smaller than the key of the node.
- All keys in the right subtree of a node are greater than the key of the node.

The restructuring operation used to move a node to the root is known as **splaying**. Depending on the position of the node, different rotations are performed. After performing the required rotations repeatedly, the accessed node becomes the root of the Splay Tree. The basic operations supported by a Splay Tree include:

- **Insertion:** Insert the node according to the BST property and then splay the newly inserted node to the root.
- **Search:** Search for the required key using the BST search procedure. If the key is found, splay that node to the root.
- **Deletion:** Splay the node to be deleted to the root, remove it, and appropriately join the remaining left and right subtrees.

Although an individual Splay Tree operation may require $O(n)$ time in the worst case, a sequence of operations has an **amortized time complexity of $O(\log n)$** per operation.



Algorithm

Algorithm 1: Splay Operation

1. Start.
2. Identify the node x that has to be moved to the root.
3. If x is already the root, no rotation is required.
4. If the parent of x is the root, perform a **Zig** rotation:
 - Perform right rotation if x is the left child.
 - Perform left rotation if x is the right child.
5. If x and its parent are both left children, perform a **Zig-Zig** rotation using two right rotations.
6. If x and its parent are both right children, perform a **Zig-Zig** rotation using two left rotations.
7. If x is a right child and its parent is a left child, perform a **Zig-Zag** rotation using a left rotation followed by a right rotation.
8. If x is a left child and its parent is a right child, perform a **Zig-Zag** rotation using a right rotation followed by a left rotation.
9. Repeat the above rotations until x becomes the root.
10. Stop.

Algorithm 2: Insertion in Splay Tree

1. Start.
2. Create a new node containing the given key.
3. If the tree is empty, make the new node the root.
4. Otherwise, compare the new key with the current node.
5. If the key is smaller, move to the left subtree.
6. If the key is greater, move to the right subtree.
7. Continue until an appropriate empty position is found.
8. Insert the new node at that position according to the BST property.



9. Perform the splay operation on the newly inserted node. The newly inserted node becomes the root.
10. Stop.

Algorithm 3: Search Operation

1. Begin searching from the root node.
2. Compare the required key with the current node.
3. If both keys are equal, the element is found.
4. If the required key is smaller, search the left subtree.
5. If the required key is greater, search the right subtree.
6. If the key is found, perform the splay operation on that node. Move the accessed node to the root.

Program Code

Write the Python program for implementation of the Splay Tree below:

Output

Paste or write the output obtained after executing the program below:

Result

Thus, a Splay Tree was successfully implemented and its basic operations such as insertion, searching, and splaying were studied.

Post-Lab Questions

1. What is a Splay Tree? How does it differ from a normal Binary Search Tree?
2. Explain the purpose of the splaying operation in a Splay Tree.
3. Explain the **Zig**, **Zig-Zig**, and **Zig-Zag** rotations with suitable examples.
4. Consider the following sequence of keys:

10, 20, 30, 40, 50

Insert these keys one by one into an initially empty Splay Tree. Draw the tree after each insertion.

5. Consider a Splay Tree containing the elements

10, 20, 30, 40, 50, 60.

Explain how the structure of the tree changes when the key 20 is searched.

6. What happens when an element that is not present in the Splay Tree is searched?

Problem statement

Binary Tree Right Side View

Medium

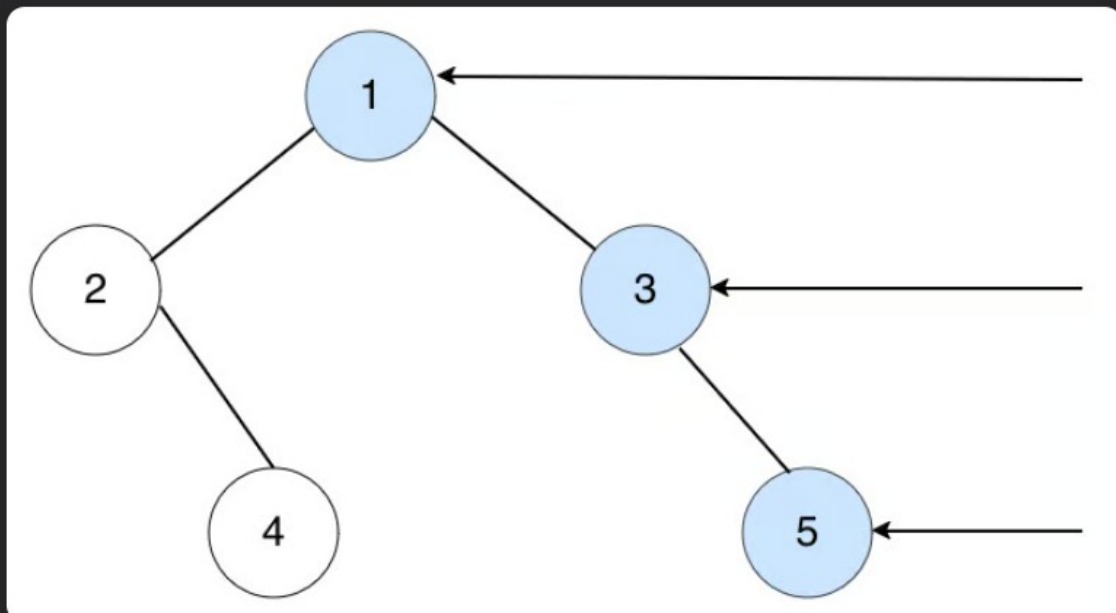
Topics

Company Tags

Hints

You are given the `root` of a binary tree. Return only the values of the nodes that are visible from the right side of the tree, ordered from top to bottom.

Example 1:



Count Good Nodes in Binary Tree

Medium

Topics

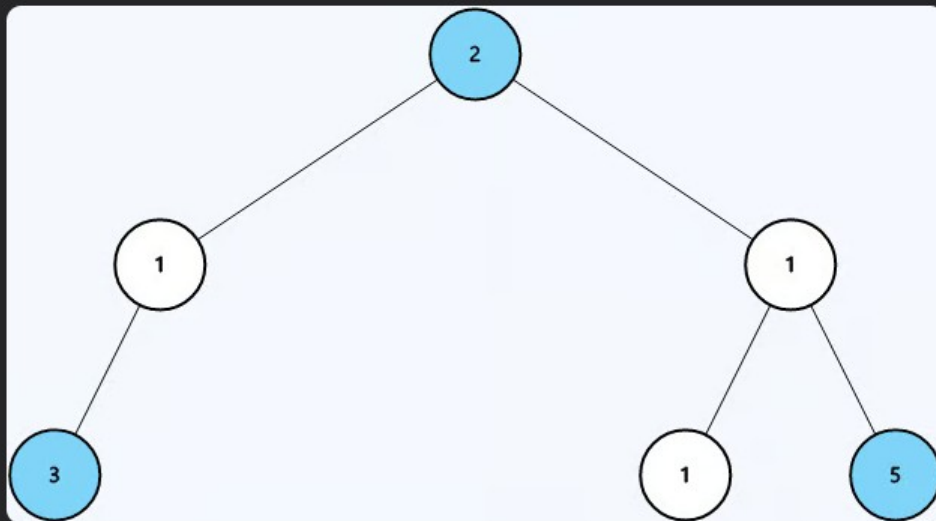
Company Tags

Hints

Within a binary tree, a node x is considered **good** if the path from the root of the tree to the node x contains no nodes with a value greater than the value of node x .

Given the root of a binary tree $root$, return the number of **good** nodes within the tree.

Example 1:



Input: $root = [2, 1, 1, 3, null, 1, 5]$

Output: 3

References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
2. Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser, *Data Structures and Algorithms in Python*, Wiley.